

Standard Library Functions

C's standard library contains many features and functions which we haven't seen.

We've seen many of `printf`'s formatting capabilities, but not all. Besides format specifier characters for a few types we haven't seen, you can also control the width, precision, justification (left or right) and a few other attributes of `printf`'s format conversions. (In their full complexity, `printf` formats are about as elaborate and powerful as FORTRAN `format` statements.)

A `scanf` function lets you do "formatted input" analogous to `printf`'s formatted output. `scanf` reads from the standard input; a variant `fscanf` reads from a specified file pointer.

The `sprintf` and `sscanf` functions let you "print" and "read" to and from in-memory strings instead of files. We've seen that `atoi` lets you convert a numeric string into an integer; the inverse operation can be performed with `sprintf`:

```
int i = 10;
char str[10];
sprintf(str, "%d", i);
```

We've used `printf` and `fprintf` to write formatted output, and `getchar`, `getc`, `putchar`, and `putc` to read and write characters. There are also functions `gets`, `fgets`, `puts`, and `fputs` for reading and writing lines (though we rarely need these, especially if we're using our own `getline` and maybe `fgetline`), and also `fread` and `fwrite` for reading or writing arbitrary numbers of characters.

It's possible to "un-read" a character, that is, to push it back on an input stream, with `ungetc`. (This is useful if you accidentally read one character too far, and would prefer that some other part of your program read that character instead.)

You can use the `ftell`, `fseek`, and `rewind` functions to jump around in files, performing random access (as opposed to sequential) I/O.

The `feof` and `ferror` functions will tell you whether you got EOF due to an actual end-of-file condition or due to a read error of some sort. You can clear errors and end-of-file conditions with `clearerr`.

You can open files in "binary" mode, or for simultaneous reading and writing. (These options involve extra characters appended to `fopen`'s mode string: `b` for binary, `+` for read/write.)

There are several more string functions in `<string.h>`. A second set of string functions `strncpy`, `strncat`, and `strncmp` all accept a third argument telling them to stop after `n` characters if they haven't found the `\0` marking the end of the string. A third set of "mem" functions, including `memcpy` and `memcmp`, operate on blocks of memory which aren't necessarily strings and where `\0` is not treated as a terminator. The `strchr` and `strrchr` functions find characters in strings. There is a motley collection of "span" and "scan" functions, `strspn`, `strcspn`, and `strpbrk`, for searching out or skipping over sequences of characters all drawn from a specified set of characters. The `strtok` function aids in breaking up a string into words or "tokens," much like our own `getwords` function.

The header file `<ctype.h>` contains several functions which let you classify and manipulate characters: check for letters or digits, convert between upper- and lower-case, etc.

A host of mathematical functions are defined in the header file `<math.h>`. (As we've mentioned, besides including `<math.h>`, you may on some Unix systems have to ask for a special library

containing the math functions while compiling/linking.)

There's a random-number generator, `rand`, and a way to "seed" it, `srand`. `rand` returns integers from 0 up to `RAND_MAX` (where `RAND_MAX` is a constant #defined in `<stdlib.h>`). One way of getting random integers from 1 to `n` is to call

```
(int)(rand() / (RAND_MAX + 1.0) * n) + 1
```

Another way is

```
rand() / (RAND_MAX / n + 1) + 1
```

It seems like it would be simpler to just say

```
rand() % n + 1
```

but this method is imperfect (or rather, it's imperfect if `n` is a power of two and your system's implementation of `rand()` is imperfect, as all too many of them are).

Several functions let you interact with the operating system under which your program is running. The `exit` function returns control to the operating system immediately, terminating your program and returning an "exit status." The `getenv` function allows you to read your operating system's or process's "environment variables" (if any). The `system` function allows you to invoke an operating-system command (i.e. another program) from within your program.

The `qsort` function allows you to sort an array (of any type); you supply a comparison function (via a function pointer) which knows how to compare two array elements, and `qsort` does the rest. The `bsearch` function allows you to search for elements in sorted arrays; it, too, operates in terms of a caller-supplied comparison function.

Several functions--`time`, `asctime`, `gmtime`, `localtime`, `asctime`, `mktime`, `difftime`, and `strftime`--allow you to determine the current date and time, print dates and times, and perform other date/time manipulations. For example, to print today's date in a program, you can write

```
#include <time.h>

time_t now;
now = time((time_t *)NULL);
printf("It's %.24s", ctime(&now));
```

The header file `<stdarg.h>` lets you manipulate variable-length function argument lists (such as the ones `printf` is called with). Additional members of the `printf` family of functions let you write your own functions which accept `printf`-like format specifiers and variable numbers of arguments but call on the standard `printf` to do most of the work.

There are facilities for dealing with multibyte and "wide" characters and strings, for use with multinational character sets.

Read sequentially: [prev](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)